Project Executable Files

Date	04-July-2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	3 Marks

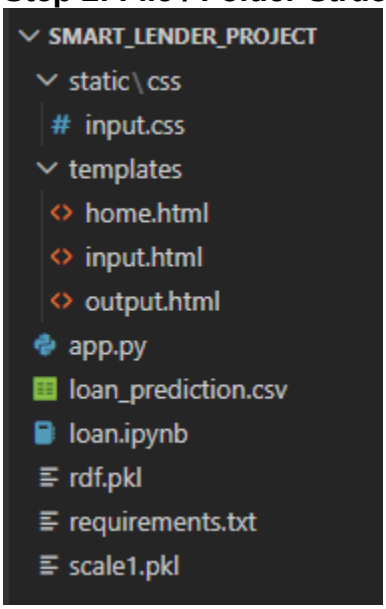
Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	NA
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure



Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	**NA** (Configured for standard standalone local execution)
Login Credentials (Demo)	NA (Direct access via portal console; no user authentication blocks required)
Platform/ Hosting Provider	Local Workspace Engine (`localhost`)
Repository Link	https://github.com/s-ath-vika/Smart-Lender
Demo Video Link	https://drive.google.com/file/d/1ByFDRB3kZxZkwHOVY0GOVj2o_m5aUTrI/view?usp=sharing

Step 4: Run Instructions

Local Execution Guidelines:

1. Install Required Project Dependencies Open your command prompt terminal inside the root project directory and execute the following line to install all required compatible libraries:

```
C:\Users\...\anaconda3\python.exe -m pip install -r requirements.txt
```

2. Generate Model and Scaler Binaries

- Open the workspace directory inside **VS Code**.
- Select and open the `loan.ipynb` file.
- Set your Jupyter execution kernel to **`base (Python 3.12.4)`.**
- Execute **Run All** cells to balance the dataset with SMOTE, train the Random Forest framework, and save out `rdf.pkl` and `scale1.pkl`.

3. Launch the Local Web App Server Run the primary Flask script through your command line console interface:

```
C:\Users\...\anaconda3\python.exe app.py
```

4. Access the Production UI Once the server prints the host confirmation, launch any modern web browser and navigate directly to:

```
http://127.0.0.1:5000
```

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Local Server Scoping: Application only binds locally to a single development loop, preventing external remote web connections.	Status: Intentional. The workspace is structured specifically for localized testing and presentation workflows.
2	Session Volatility: User input features and generated underwriting results are processed on-the-fly and vanish once the page is refreshed or closed.	Workaround: Fixed. Built-in HTML dropdown elements and strict backend data-type parsing filter input before execution.
3	Strict Numeric Input Constraints: Submitting missing data structures or unmapped alphanumeric strings via raw API requests causes processing halts.	Workaround: Fixed. Built-in HTML dropdown elements and strict backend data-type parsing filter input before execution.